

Final Project

Software Engineering - UATX - Spring 2026

Due	Day of lecture 10.2 - everything live and shipped before class starts
Team size	2 or 3 people. No solos, no fours.
Stack	FastAPI backend, React/TypeScript frontend, Postgres database (hosted, not local), deployed to a public cloud URL. Specific hosting and DB provider are your call - Railway + Supabase is the recommended default we walked through in class.
Submit	Public GitHub repo, public cloud URL, one short README. Details below.

The project

The final project is three weeks of building something new, in a team of two or three, deployed to the internet, that you'll demo on the last day of class. It's the same kind of work you've done in the individual assignments - backend, database, frontend, real user flow - but with three differences: it's your idea, you're in a team, and there's no new assignment landing every week to keep you moving.

What I want by the end is a working web app that someone who's never seen it before could land on, sign up for, and actually use to do whatever it is the app does. Bronze is the floor - it's the version that has to exist for the project to count - but it's not where you should be aiming. A polished silver project beats an unfinished gold one, and either beats a barely-deployed bronze.

What counts as a final project

The shape of the assignment is open - I want you picking something you actually want to spend three weeks on. But there are a handful of invariants that every project has to satisfy, regardless of what you build:

1. **A FastAPI backend exposing a JSON API.** At least six or seven real endpoints. SQLAlchemy or raw SQL is your call.
2. **At least one piece of nontrivial logic in your app.** Not just CRUD on a single table - a ranking, a recommendation, a scheduling decision, a real multi-step state machine, a non-trivial query that aggregates across tables, something where you have to think about ordering, edge cases, and invariants. The bar I'm checking: could a coding agent produce the right answer from a one-line description, or did you have to actually design it? If it's the former, that's CRUD - find something else. This piece can live in an endpoint, in a frontend reducer, in a background job, wherever fits the app. Describe it specifically in your README so I know where to look.
3. **A Postgres database, hosted in the cloud** - meaning a real managed Postgres your deployed app connects to over the network, not a SQLite file or a local Postgres on your laptop. Supabase is the easy default and is what we walked through in class; Neon, Render's Postgres, AWS RDS, anything else managed and cloud-hosted is also fine. Real foreign keys and the constraints (NOT NULL, UNIQUE, length limits) that your API enforces.
4. **A React + TypeScript frontend** served as the primary UI. The deliverable is a web app you open in a browser, so no "and there's also a CLI we use for the real stuff."
5. **Real user identity.** People sign up, log in, and your app knows who they are across pages and across refreshes. Supabase Auth handles this for you and I'd really recommend it, or Google sign-in via something like Auth0/Clerk if you'd rather - the point is just to not roll your own from scratch, which is a way to spend the whole three weeks on the login page. The `X-Username` header trick from A2 and A4 is NOT enough here.
6. **The value has to come from multiple users interacting.** A solo todo list doesn't count, and a solo journal doesn't count either - there needs to be some reason the app exists because there are other people on it (a feed, a shared space, a leaderboard, comments, a marketplace, a directory, something). A useful test: if you could rebuild your project as a single-user desktop app with nothing lost, it's not the right scope.
7. **A real test suite and CI.** A tests directory with ~8-10 tests covering your main endpoints and at least one frontend test, plus a GitHub Actions workflow that runs them on every push to main and gates deploy on passing tests. Coding agents write tests easily, so the bar here is not "did you write all the tests yourself" - it's "do the tests cover the important paths, do they catch real regressions, and does CI fail when they fail." We'll walk through GitHub Actions setup in class.

8. **Deployed to the cloud at a public URL.** Anyone with the URL can sign up and try the app, no "request access" wall. Railway is the easy default and is what we walked through in class; Fly.io, Render, Vercel (for the frontend with a separate backend service), or anything else that gives you a real public URL works too. When I grade, I'm going to open the URL in my browser and use the app, so it has to actually work that way.
9. **Source on a public GitHub repo** at `github.com/<someone-on-your-team>/<project-name>` . Real commit history from multiple team members - I will look at `git log` .
10. **A README** that I can read in five minutes and understand what you built, who built which parts, what tier you're claiming, and where to find your nontrivial-logic piece. Details in the README section below.

That list IS the spec. Everything else - what domain you pick, what features you build, how it looks, what the data model is - is yours to figure out.

A worked example: BookCircle

To make the scope concrete, here's a project you could build called BookCircle. You don't have to build BookCircle - in fact please don't, since we'd end up with half the class submitting the same project - but it's a useful anchor for thinking about scope, because it satisfies every invariant above and has a clean bronze / silver / gold version. When you're trying to figure out if your team's idea is the right size, a good question is whether the bronze tier of your project is about as much work as the bronze tier of BookCircle.

BookCircle, the pitch

A small private reading club. You and a few friends create a "circle." Inside the circle, people post what they're currently reading, share short reviews when they finish a book, and comment on each other's reviews. You can click into anyone's profile to see their reading history. The whole point is to know what your friends are reading and to surface books worth reading next.

BookCircle at bronze

Bronze means the basic version of the app works end to end, deployed, with all the invariants from the section above. For BookCircle, that breaks down as roughly:

- **Data model.** Tables for users (a profile row paired with the Supabase Auth user), circles, memberships (which users are in which circles), reviews (a post: book title, author, rating, body text, written by a user inside a circle), and comments. Real foreign keys.
- **API.** Endpoints for: list my circles, create a circle, join a circle by invite code, list reviews in a circle (the feed), post a new review, fetch a single review with its comments, post a comment, fetch a user's profile and recent reviews. Around eight endpoints. Auth-gated using Supabase's JWT - your FastAPI checks the token on every request.
- **The nontrivial piece.** Example: a "top books in this circle" view that aggregates ratings across all reviews of the same book (matched by normalized title + author), filters to books with at least N ratings, sorts by average rating with a sensible tiebreaker, and excludes books the viewing user has reviewed themselves. Real cross-table aggregation, real edge cases (what counts as "the same book"? what's the tie-breaker between two 4.5-star books?), real design decisions. The team should be able to walk me through how it works at the demo.
- **Frontend.** Pages for sign-up/log-in, my-circles, inside-a-circle (the feed), post-a-review, single-review-with-comments, a user's profile, top-books-in-the-circle. Every fetch has loading + error states. The "who am I" persists across refresh because Supabase Auth keeps the session.
- **Tests + CI.** ~8-10 pytest tests on the API (the feed endpoint, the post-review endpoint, the top-books query - test the happy path AND at least one edge case for each), plus one Vitest test exercising a frontend component. GitHub Actions workflow runs them on push, blocks merge to main on failure, deploys only if green.
- **Deployed.** Live at `bookcircle.up.railway.app` or whatever URL your cloud provider gives you. I sign up with a fresh email, create a circle, post a review, leave a comment on my own post, and it all works.

That's the bronze version of BookCircle. It's also a real, usable product - my family could actually use BookCircle at this tier without feeling like they're poking at a school project, which is kinda the level of done you're aiming for. Bronze this semester is about shipping something that actually works, not about packing in a lot of features.

BookCircle at silver

Silver means bronze plus a layer of real architectural and product decisions on top. For BookCircle, possible silver moves:

- **A second nontrivial piece.** Example: a "what should I read next" suggestion that looks at books two or more people in your circle have rated four stars or better, that

you haven't reviewed yet, ranked by how strongly the people closest to your taste rated them (similar to bronze's "top books" but personalized rather than circle-wide). Sits next to the bronze top-books query in your codebase, but with a different design problem (defining "taste similarity") behind it.

- **Search.** A search box on the feed that hits a `GET /circles/{id}/reviews?q=...` endpoint and matches book title, author, or review body.
- **Optimistic updates.** Posting a new review or comment shows up in the UI instantly, before the server has confirmed, and rolls back if the request fails.
- **Real routing.** Bookmarkable URLs - `/circles/abc` , `/circles/abc/reviews/42` , `/users/jenny` . The browser back button works. Refreshing keeps you where you are.
- **A visual design that isn't unstyled HTML.** A coherent type scale, a real color palette, deliberate spacing. Doesn't have to be pretty - it has to look like someone made choices.
- **An e2e-ish test.** One Playwright (or similar) spec that signs up a user, creates a circle, posts a review, and verifies it appears on the feed - exercising the full happy path of the app from the browser.

BookCircle at gold

Gold means silver plus a phone-friendly app, one of {real-time, end-to-end tests, real visual design}, and at least two nontrivial features you came up with. For BookCircle, examples of how each piece might land:

The "pick one" piece - one of these three, your choice:

- **A real-time-ish feed.** When someone in your circle posts a review, my open feed shows it within five seconds without a manual refresh. Polling with `setInterval` is fine - it's the easiest path. Server-sent events or websockets are cooler but more work. Whichever you pick, explain in the README why it fits.
- **An end-to-end test suite.** Multiple Playwright specs covering at least three distinct flows - sign up + join a circle, post a review and see it on the feed and on your profile, an edge-case like "user not in this circle can't see its reviews." Wired into the bronze CI pipeline so the whole thing has to go green before deploy.
- **A visual design that has a point of view.** Not just "looks fine" - a deliberate aesthetic. Reads well on a phone-width browser too. Plain CSS, Tailwind, a component library - your call, no requirement either way.

Plus at least two custom features. These have to involve real state and real interaction. Some examples of what that could look like for BookCircle:

- **"What should I read next?"** A view that surfaces books two or more people in your circle rated four stars or more, that you haven't reviewed yet. Forces a real cross-table query rather than just listing things.
- **Reading streaks.** The app tracks how many days in a row each user has logged at least one reading-related action (a review, a comment, a "currently reading" update), and shows it on their profile. Real time-window queries, real edge cases around timezones.
- **Friend recommendations within the circle.** "These three people in your circle have rated three or more of the same books as you" - cheap collaborative filtering, but the data join is real.
- **Reading goals and progress.** A user sets a yearly book goal; the app shows their pace against the goal and surfaces it on the feed when they hit milestones. Real cron-ish thinking even if the implementation is "check on each render."

Two of those, or two of your own - the point is they should feel like product features, not engineering exercises.

Other directions

BookCircle is one project among many that hit the invariants. If reading clubs don't excite you, here are five other directions, each of which would be a reasonable final project at this scope. None of these are spec'd out - they're sketches. The point is to show the range of things that count:

- **Sprint Club.** A workout/running tracker for a small group. Log a workout (type, duration, distance, date), see your own history, see a weekly leaderboard for the group. Silver could add reactions/kudos and a real chart of your distance over time. Gold could add a real-time leaderboard during a group challenge.
- **UATX Study Sessions.** Students post when and what they want to study; other students join. Each session has a chat thread that lives until the session ends. Silver could add recurring sessions and filtering by course. Gold could add a calendar view.
- **Splitwise for one trip.** A group going on a trip creates a "trip," logs expenses as they happen, and the app keeps a running tally of who owes whom. Silver could add receipts-as-images and per-category breakdowns. Gold could add real-time updates as people add expenses from their phones during the trip.
- **Potluck Planner.** The host creates an event, sets a few categories (appetizer, main, dessert, drinks). Guests sign up to bring specific things, and the app prevents double-bookings on the unique items. Silver could add comments per signup and a printable summary. Gold could add real-time as guests sign up.

- **The UATX student marketplace.** Tiny Craigslist for the campus - post things you want to give away, swap, or sell. Search by category, message the poster. Silver could add saved searches. Gold could add image uploads (Supabase Storage handles this) and a real conversation thread.

If your idea isn't on this list and isn't BookCircle, that's good. You're welcome to grab me before or after class if you want a gut check on scope. The easiest way to sink this project is to go too big - "we're building Instagram, but better" - and then arrive at week 10 with a half-built profile page. Pick something where silver is comfortably achievable in three weeks of work; bronze is the floor, not the target.

Setup and stack

The language/framework layer is fixed - you've spent the whole semester learning this exact stack, and switching now would just burn the time you need for actually building. The hosting and database provider are flexible, as long as they're real cloud services. Here's the layout:

- **Backend:** FastAPI (Python), the same as A2 and A3.
- **Frontend:** React + TypeScript + Vite, the same as A4.
- **Database:** Postgres, hosted somewhere in the cloud. Supabase is the easy default we walked through in class; Neon, Render Postgres, AWS RDS, etc. all work too. SQLAlchemy is fine, raw SQL is fine. Local SQLite is NOT fine - the database has to live in the cloud, separate from your app.
- **Auth:** Real auth that survives refreshes. Supabase Auth is the recommended path because it's basically free, but Google sign-in (via Auth0, Clerk, or similar) is also great if you'd prefer. Rolling your own JWT setup from scratch is allowed if you really want to, but I'd think hard before signing up for that. The `X-Username` header trick from A2 and A4 isn't enough.
- **Hosting:** Anywhere you can get a public URL that stays up - Railway is the easy default we walked through in class, Fly.io / Render / similar are fine. The architectural trick from class (FastAPI serves the built React app at `/`, handles `/api/*` for the API, one service, one URL, no CORS) applies regardless of provider.
- **Source:** Public GitHub repo, one per team. Real commits from each member - I will look.

If you go with the Railway + Supabase defaults: sign up for the GitHub Student Developer Pack at education.github.com/pack if you haven't, it covers Railway's monthly fee.

Supabase's free tier is way more than enough. If you pick a different provider, just make sure your README explains the choice in a sentence so I'm not hunting for it.

Tiers

The bronze/silver/gold structure is the same one you've used all semester. Each tier is a calibration of what "done" means, NOT a checklist of features - a project with a small set of features all working perfectly is a stronger submission than a project with twice as many features that mostly half-work. When grading, I'm really asking whether the thing actually works, whether the team made real decisions, and whether the experience holds together.

Bronze. Every invariant from "What counts as a final project" is satisfied: the app is live at a public cloud URL, anyone can sign up, real user identity persists across refresh, multi-user interaction works end to end, every fetch has visible loading and error states, the database has multiple related tables with real constraints, the API has at least six or seven endpoints, at least one piece of your app's logic is the nontrivial-thinking thing (not just CRUD), tests cover the main paths and run in CI on every push to main with deploy gated on green, and the README is there and complete. The whole thing works in a fresh browser session I've never used before, without me having to fiddle.

Silver. Bronze plus all of the following:

- **A second nontrivial piece of logic** beyond the bronze one. A second algorithm, a second cross-table query, a second state machine, something with similar design weight to the bronze one. Describe it in the README alongside the first.
- **Optimistic updates** on at least one action, with rollback on failure.
- **Real client-side routing** with bookmarkable URLs. Back button works.
- **A visual design that someone made choices about.** Not unstyled forms.
- **Test coverage that goes beyond the bronze floor.** Either a few more tests across new silver behavior, or at least one end-to-end-ish test that exercises a full user flow (sign up, do the main action, see the result).

Gold. Silver plus **all three** of the following:

- **The app works well on a phone.** Every screen is usable and looks right at phone width - no horizontal scrolling, no tap targets you can't hit, no layout that only holds together on a desktop monitor. I will open your app on my phone.
- **At least one** of:
 - **Real-time-ish updates.** When user A does the relevant action, user B's open view reflects it within ~5 seconds without a manual refresh. Polling is fine. Explain in the README why polling-vs-push fits.
 - **A real end-to-end test suite.** Multiple Playwright (or similar) specs covering at least three distinct user flows - sign-up + onboarding, the main create-and-interact action, and one edge-case path like "unauthorized user tries to do something they shouldn't." Runnable with one command, and ideally wired into CI alongside the bronze pytest tests so the whole pipeline goes green before deploy.
 - **Visual design with a real point of view.** Coherent type scale, deliberate color choices, decent spacing, holds up at mobile width. Plain CSS, Tailwind, a component library, anything is fine - I'm grading the result, not the tool.
- **At least two nontrivial features you came up with.** These have to involve real state and real interaction, not just a styling tweak or a new copy of an endpoint you already had. The BookCircle gold section below gives the flavor of what counts. Describe each one in the README in a sentence or two.

One thing on how I'll grade: I'd rather see a project with five solid, polished features than a project with eight half-working ones. Features that work get points; features that crash, glitch, or only-sometimes-do-the-right-thing don't, and they also pull down the overall impression of the project, so it's a double penalty. Cutting scope is a real skill, and being willing to drop features you're attached to is usually what makes a 3-week project finishable rather than half-built.

Timeline and milestones

Three weeks is not as much time as it sounds. It's easy to spend a week deciding what to build, another week scaffolding, then realize there's only a week left and nothing's deployed yet. The milestones below are there to push back on that.

By lecture 8.1 (the lecture after this one)

- **Team locked.** Two or three names. Email them to me if you haven't told me yet.
- **Rough idea agreed on.** Something like "BookCircle: a small reading club where friends post reviews and see what each other are reading." A sentence is enough; more is fine.

By lecture 9.1

- **The data model is real and the app is deployed.** Your actual tables (with foreign keys and constraints) live in your cloud Postgres. You've written and deployed at least three of your real API endpoints. Your frontend has at least one real page that hits a real endpoint and shows real data. The whole thing is reachable at a public URL.
- **CI is running.** A GitHub Actions workflow runs your pytest tests on every push, even if "your pytest tests" right now is just two or three. Adding more tests is just adding files to the directory at that point.
- Halfway point. If you're not here yet by 9.1, you're welcome to come find me.

Day of lecture 10.2

- **Everything is due before class starts.** Deployed app at a public URL, source pushed to `main`, README in the repo root, README's "tier targeted" line filled in.
- **Demos in class.** Each team will have 5-7 minutes to show the app live. Plan to walk through your main user flow, point out one or two things you're proud of, and answer a question or two about how it's built.
- **No extensions.** The demo is whatever is at the URL when I open it. If your deploy has been broken since Sunday, that's the demo. Plan accordingly.

What to submit

1. **A public GitHub repo.** One per team. Pushed to the default branch (probably `main`). All team members have made real commits.
2. **A live URL.** Linked from the top of your README. Loads in a fresh browser. Sign-up works end to end.
3. **A README at the repo root.** See requirements below.
4. **An email to me** by 11:59pm the day before 10.2 with: team members, the GitHub URL, the live URL, and the tier you're claiming. If the README has all this, paste it into the email.

README requirements

The README is what I read first. Keep it short and direct. It should contain:

1. **One-paragraph pitch.** What does your app do, who is it for. Same kind of one-sentence framing as the BookCircle pitch above, plus a sentence or two of context.
2. **Live URL and GitHub URL.** At the top so I can click through immediately.
3. **Tier targeted.** Bronze, silver, or gold. Don't claim a tier you didn't hit - I'll regrade down.
4. **Team members and what each person did.** One short bullet per person. Don't game this - I'll be cross-referencing with the git log.
5. **Where your nontrivial-logic piece lives.** File path + function name + a sentence on what it does and what the design decision was. Silver should mention both pieces. Gold should mention all three (bronze + silver + the gold "pick one" if it's the e2e or visual one, or a brief description of how the gold pick differs from the silver/bronze pieces).
6. **Three or four design decisions you made.** Real ones, with the reasoning. "We chose Supabase Auth over rolling our own because..." / "We modeled circles as separate from organizations because..." / "We used polling instead of websockets because..."
7. **Where your agents helped most and where you had to push back.** One paragraph. Not for grading - I want to read these as a class to see how it's going. Specifics are great: "Claude wanted to denormalize the reviews table so each review had an embedded user_name, and we had to push it toward a foreign key + join because we knew users could rename themselves."
8. **How to run locally and how to run tests.** The install commands, the env vars I'd need if I were spinning up my own copy, and the one command that runs your test suite. Don't be exhaustive - one block is enough.
9. **Gold only:** a sentence on which of the three "pick one" options you went with, and a sentence or two on each of your custom features.

A note on teams

This is the first time most of you will have built software in a team, and the team dynamic part is often harder than the technical part. A few patterns to watch for:

Divide work along seams, not tasks. "You do the backend, I do the frontend" works for the first few days and then falls apart, because every real feature ends up needing both. Better is to have one person own reviews end-to-end (the table, the endpoints, the frontend pages, the styling), another person own comments end-to-end, a third person own the auth + circles bit. Each person becomes the "ask them" person for their area, and you'll still help each other across boundaries - but ownership stays clean and reviews don't bottleneck on one person.

Consider feature branches. With 2 or 3 people on the same codebase, it's worth thinking about whether feature branches are useful for you, vs. everyone just pushing to `main`. Branches make it easier to figure out which change caused something to break in week 3, but they also add overhead. PR-based code review is not something I'd push you on - you're early enough in your careers that it's hard to know what to look for in someone else's code. Pick whatever workflow keeps the team unblocked.

Sync often, in person if you can. Set a standing 15-minute meeting at least twice a week, more if you can swing it. It's easy for two teammates to work in parallel on the same thing for three days because they didn't compare notes on Tuesday - the more you talk to each other, the less of that you get.

A note on coding agents

The AI rules are the same as the individual assignments. Use Claude / Cursor / whatever you've been using. The grading I'll be doing on the demo day will involve me asking you about your code - which fetch lives where, why a particular table has the columns it does, what happens when two users post at the same time - and the test is whether you can answer. If you and your team have shepherded the agent through the project, you'll be able to answer. If you've taken the first thing it produced and shipped it, you won't.

One specific thing worth pushing on: the agent will happily generate code that works in the happy path and ignore everything else. Loading states, error states, what happens when two users do the same action at once, what happens when the form is submitted twice fast, what happens when someone hits the browser back button mid-action. Push your coding agent on these. Or, write the scaffolding for the error states yourself first, then let the agent fill in the success path - that ordering tends to produce much better results than "build the feature, then make it robust."

The other half of this is README question 6 above. I read those as a batch, and I expect them to be some of the most useful artifacts of the semester for me - I get to see, across the whole cohort, where coding agents are reliably good and where they reliably need a human

leaning on them. Be honest about both. Nobody's grading you on how many times you pushed back, so don't manufacture stories.

Grading

Bronze	Satisfactory: a working, deployed, multi-user app that hits every invariant
Silver	Good: bronze plus real architectural choices, routing, optimistic updates, tests, and visual design that's been thought about
Gold	Excellent: silver, plus one of (real-time updates / end-to-end tests / design with a point of view), plus at least two nontrivial features you came up with

Style and quality matter more here than they did on the individual assignments - not necessarily in the grade, but in how much the decisions cost you. A messy codebase on a three-week team project is something you'll feel the pain of directly, well before I ever grade it. Things that land in this bucket: the README being readable, the repo being organized rather than a pile of files in one folder, sensible commits from real people rather than one big "final commit" from one teammate, a visual design that looks like someone made choices, code that a teammate could pick up. The team process piece specifically: real distribution of work across team members, no single hero carrying it.

One thing that's new at team scale: you haven't needed a `CLAUDE.md` for the individual assignments, but you'll likely want one here. It's the place to write down the conventions your team decides on - directory layout, naming, how you structure endpoints, whatever - so that Claude picks them up the same way for every developer, in every session. Without it, each teammate's agent quietly invents its own conventions and you spend the back half of week 3 reconciling them.

What I want most is that on the day of 10.2, when you're standing in front of the class with your live URL up on the screen and you click through the main flow, the room feels like "yeah, that's a thing I'd actually use." If you get there, the rest of the grade pretty much takes care of itself.